I need a script that helps deteceta texts on models that the other script cant detect can you do a deep search into something l=that would help]

Now I have everything I need. The research is solid. Here's the full spec:

The problem is specific and well-defined from your own logs: `djinn-model-text-engrave` uses **bounding box** to find surfaces, but the Camood job proved that bounding box is wrong — the actual flat face was 2.5mm inside the bounding box crown, found only by ray-casting. The script you need is `djinn-detect-surfaces` — a pre-flight surface scanner that finds **every engravable face by ray-cast and curvature, not bounding box**, and hands coordinates to `djinn-model-text-engrave` instead of letting it guess.

What the Current Tool Can't Detect

From the June 4 report and TASK-062 , three categories of surfaces currently fail:

Failure Mode	Why It Fails	Example
Flat face set back from bounding box	Tool uses `Y_max` / `Z_max` as face position — overshoots by 2.5mm	Camood tank back face
Flat **vertical** faces on any axis	`djinn-model-text-engrave` has no `--flat-panel` mode — vertical rays miss them entirely	Any wall-engraved piece
Curved convex surfaces	TASK-062 explicitly marks curved surface analysis as Phase 2 — currently returns "warn and workaround" with no actual data	Cylinders, domes, organic bodies

The fix is a **standalone surface detection pre-script** that does the geometry work before `djinn-model-text-engrave` touches anything.

The Core Technique: Ray-Cast Surface Finding

The Camood session proved the correct method : cast rays perpendicular to the expected face direction from a grid of points, take the median of all hit positions. This gives the **actual surface position** rather than the bounding box edge.

Trimesh's `mesh.ray.intersects_location()` is the right API. You pass an array of ray origins and directions, it returns hit coordinates and face indices. For a flat back wall, you grid-sample in XZ, shoot rays in the +Y direction, and median the hit Y values — that's the face. [1] [2]

```
ray_origins = [[x, far_y, z] for x,z in grid]
ray_directions = [[0, -1, 0]] * N
hits = mesh.ray.intersects_location(ray_origins, ray_directions)
face_y = np.median(hits[:,1])   # The real face — not bbox
```

This is exactly what `engrave_tank.py` did manually . The new script makes it automatic for **all 6 faces** in one pass.

The Second Technique: Curvature-Based Zone Detection

For curved surfaces, bounding box and ray median both lie — the face isn't flat so there's no single hit position. Trimesh has built-in discrete Gaussian and mean curvature measures via `trimesh.curvature.discrete_mean_curvature_measure()` and `discrete_gaussian_curvature_measure()`. The workflow:[3] [4]

1. Compute vertex-level Gaussian curvature across the whole mesh — `vertex_defects()` gives a scalar per vertex, 0 = flat, positive = convex, negative = concave[3]
2. Threshold near-zero curvature vertices → these are the flat zones even on a curved body
3. Cluster the flat-zone vertices by normal direction → each cluster is an engravable region
4. For non-flat zones, compute the curvature radius from the mean curvature value — TASK-062 specifies the constraint: curvature radius must be large enough that text distortion at the edges is below threshold

This is the exact "discrete mean curvature via cotangent weights" that TASK-062 labels Phase 2 — this script builds it.

Script Design: `djinn-detect-surfaces`

What It Does

Takes an STL, no other arguments required. Outputs a JSON file that `djinn-model-text-engrave` reads instead of doing its own surface detection.

```
djinn-detect-surfaces model.stl [--output surfaces.json] [--render] [--min-area 25]
```

The Six-Direction Ray Grid

For each of the 6 cardinal directions, it fires a 20×20 grid of rays across the full bounding box face, collects hits, removes outliers (>2σ from median), and computes:

- **Actual face position** (median of hit coordinates on the face-normal axis)
- **Engravable region bounds** (convex hull of hit points projected to face plane)
- **Surface flatness** (std dev of hit positions — low = flat, high = curved or complex)
- **Hit density** (fraction of rays that hit anything — low = mostly open, not a solid face)

Any direction where hit_density > 0.7 and flatness_std < 0.5mm → confirmed flat engravable face, with real coordinates. [1]

The Curvature Pass

After the ray pass, for every surface region that failed the flatness test (std > 0.5mm), it runs the curvature analysis: [4] [3]

```
from trimesh.curvature import discrete_gaussian_curvature_measure, vertex_defects

# Per-vertex Gaussian curvature (0 = flat)
k = mesh.vertex_defects()

# Flat zone threshold: |k| < 0.01 rad/mm²
flat_vertices = np.where(np.abs(k) < 0.01)[^0]

# Cluster by normal direction → each cluster = one engravable zone
```

For each curved region that has a large-enough flat zone, it reports the zone centroid, normal, and the local curvature radius so the constraint engine can check if text will distort .

Output Format


```
{
  "file": "Camood.stl",
  "dimensions": {"x": 78.4, "y": 54.1, "z": 140.2},
  "surfaces": [
    {
      "id": "back_flat",
      "label": "Back flat panel",
      "detection_method": "raycast",
      "normal": [0, 1, 0],
      "face_position_mm": {"y": 51.553},
      "engravable_bounds": {"x_min": 12.3, "x_max": 65.1, "z_min": 55.0, "z_max": 125
      "area_mm2": 1382.4,
      "flatness_std_mm": 0.021,
      "hit_density": 0.94,
      "engravable": true,
      "engravability_score": 0.89,
      "warnings": []
    },
    {
      "id": "body_curve_front",
      "label": "Front curved body",
      "detection_method": "curvature",
      "normal": [0, -1, 0],
      "curvature_radius_mm": 31.2,
      "flat_zone_centroid": [39.2, 3.1, 88.0],
      "flat_zone_area_mm2": 420.0,
      "engravable": true,
      "engravability_score": 0.51,
      "warnings": ["Curved surface — text >20mm wide will distort at edges"]
    }
```

```
  ]
}
```

Integration With `djinn-model-text-engrave`

Add one flag to the existing tool:

```
djinn-model-text-engrave model.stl --text "Terp Tribe" --surface-data surfaces.json
```

When `--surface-data` is passed, the tool reads the JSON instead of running its own detection. The surface position, normal, and engravable bounds come from the pre-flight scan — no bounding box guessing, no missed vertical faces .

What This Fixes vs. What TASK-062 Says

Gap from TASK-062	This Script Addresses It
`djinn-model-text-engrave` has no flat-panel mode	Ray-cast pass finds all flat faces including vertical, provides real coordinates
Bounding box overshoot (Camood 2.5mm error)	Ray-cast median replaces bbox for all 6 directions
Curved surface analysis is Phase 2	`vertex_defects()` curvature pass + flat-zone clustering handles it [3]
`engrave_tank.py` standalone — not integrated	This script generates `surfaces.json` that becomes the integration bridge

One Implementation Note for Claude

The script should use `trimesh.ray.ray_pyembree` if `pyembree` is installed, `trimesh.ray.ray_triangle` otherwise. The pyembree backend is ~100× faster for large meshes (55k+ faces like the Camood). The detection pass grids 400 rays per direction × 6 directions = 2,400 total ray queries — fast either way, but the check costs nothing. [5] [2]

The `vertex_defects()` call is the slow part on high-poly meshes — if mesh has >100k faces, subsample to 50k using `trimesh.simplify_quadratic_decimation` before curvature analysis, then map results back to original using nearest-neighbor lookup. TASK-062 already specifies this pattern .

⁂

1. https://trimesh.org/ray.html
2. https://trimesh.org/trimesh.ray.ray_triangle.html
3. https://trimesh.org/trimesh.curvature.html
4. https://trimesh.org/curvature.html
5. https://github.com/mikedh/trimesh

6. https://www.sandia.gov/files/cubit/15.8/help_manual/WebHelp/mesh_generation/meshing_schemes/traditional/trimesh.htm
7. https://arxiv.org/abs/2106.01178
8. https://www.facebook.com/groups/889882791637410/posts/1699153247377023/
9. https://pmc.ncbi.nlm.nih.gov/articles/PMC12713589/
10. https://www.reddit.com/r/Fusion360/comments/15x9rjo/how_do_i_emboss_text_on_this_mesh_body_i_cant/
11. https://www.youtube.com/watch?v=VKd06fcGZt0
12. https://www.materialise.com/en/academy/healthcare/mimics-innovation-suite/video-tutorials/label-3d-printed-anatomical-models
13. https://trimesh.org
14. https://stackoverflow.com/questions/tagged/trimesh?tab=Newest
15. https://www.youtube.com/watch?v=t78T0_qSrZM
16. https://arxiv.org/html/2410.00320
17. http://arxiv.org/pdf/2412.13461.pdf
18. https://arxiv.org/pdf/2112.09045.pdf
19. http://arxiv.org/pdf/1903.08536.pdf
20. https://arxiv.org/pdf/2408.16201.pdf
21. https://arxiv.org/pdf/2311.14897.pdf
22. https://arxiv.org/pdf/2309.11248.pdf
23. https://isprs-archives.copernicus.org/articles/XLVIII-2-W4-2024/295/2024/isprs-archives-XLVIII-2-W4-2024-295-2024.pdf
24. https://www.youtube.com/watch?v=nJFRgiMRWmU
25. https://it.mathworks.com/matlabcentral/fileexchange/181273-anatomy-curvature-analysis?s_tid=FX_rc3_behav
26. https://www.grasshopper3d.com/forum/topics/curvature-of-mesh-vertex-points-via-python
27. https://towardsdatascience.com/anomaly-detection-in-manufacturing-part-3-visualize-the-results-a2afb5f61d2f/
28. http://www.ysxbcn.com/down/upfile/soft/20120106/16-p315.pdf?WebShieldDRSessionVerify=BZ182RWXlKvzKWQMcxuN
29. https://github.com/M-3LAB/awesome-3d-anomaly-detection/blob/main/README.md
30. https://researchportal.hw.ac.uk/en/publications/detection-of-salient-curvature-features-on-polygonal-surfaces/
31. https://arxiv.org/html/2606.01992v1
32. https://www.academia.edu/65153325/Automatic_recognition_of_features_from_freeform_surface_CAD_models
33. https://github.com/mikedh/trimesh/blob/main/trimesh/curvature.py
34. https://www.newswise.com/articles/3d-vision-based-anomaly-detection-in-manufacturing-a-survey
35. https://aurorahelp.zebra.com/aurorail/MILXSP4/MILXSP4HelpFiles/extfile/z_MIL_UG_registration/UserGuide/registration/Photometric_Stereo.htm
36. https://www.youtube.com/watch?v=2vtxIOOMV_A
37. https://www.mdpi.com/2313-433X/7/6/96/pdf
38. https://arxiv.org/pdf/2305.01867.pdf
39. https://isprs-archives.copernicus.org/articles/XLVIII-4-W10-2024/5/2024/isprs-archives-XLVIII-4-W10-2024-5-2024.pdf

40. https://msia.centre-mersenne.org/item/10.5802/msia.22.pdf
41. http://arxiv.org/pdf/2403.11453.pdf
42. https://discourse.threejs.org/t/unable-to-raycast-to-mesh-walls/35614
43. https://earthsim.holoviz.org/user_guide/Analyzing_Meshes.html
44. https://github.com/mikedh/trimesh/issues/1387
45. https://stackoverflow.com/questions/78824557/how-to-slice-a-3d-volume-at-a-curved-2d-surface-using-python
46. https://graphics.stanford.edu/software/trimesh/trimesh-doc.html
47. https://docs.rs/parry3d-f64/latest/parry3d_f64/shape/struct.TriMesh.html
48. https://stackoverflow.com/questions/69336886/python-trimesh-x-cross-section-produces-weird-result
49. https://trimesh.org/trimesh.ray.html
50. https://trimesh.org/section.html
51. https://github.com/mikedh/trimesh/issues/306
52. https://stackoverflow.com/questions/67162284/slice-mesh-with-trimesh
53. https://arxiv.org/pdf/2409.07454.pdf
54. https://github.com/py5coding/py5generator/discussions/620
55. http://cvlab.cse.msu.edu/project-mdr.html
56. https://www.youtube.com/watch?v=GM-WQXxJN9M